

UD2: Programación Multihilo

2.2.-Creación de Hilos

En Java existen dos formas para crear hilos.

Ambas son parte del paquete `java.lang.thread`

1.- Extendiendo de la clase Thread

```
public class MiThread1 extends Thread
```

2. – Implementando la interfaz Runnable

```
public class MiThread2 implements Runnable
```

API class Thread <http://docs.oracle.com/javase/7/docs/api/java/lang/Thread.html>

1:-La clase Thread

Forma general del cuerpo de una clase hilo mediante extensión de la clase Thread:

```
public class NombreClase extends Thread{  
    //propiedades, constructores y métodos de la clase  
    ....  
    public void run(){  
        // acciones que lleva a cabo el hilo  
        ....  
    }  
}
```

Dentro de la clase que usa a NombreClase, la siguiente es la forma del código para crear el hilo e iniciar su ejecución mediante el llamado al método start():

```
NombreClase unHilo = new NombreClase();  
.  
.  
unHilo.start();
```

Programación de Servicios y Procesos

2.2.-Creación de Hilos

```
public class MiThread1 extends Thread {           Ejemplo extends Thread 1
    public void run() {
        System.out.println("Soy el hilo creado. Voy a contar.");
        for (int x=1; x<=10; x++)
            System.out.println(x);
    }
}

public class Principal {
    public static void main(String[] args) {
        MiThread1 t1 = new MiThread1();
        t1.start();
        System.out.println("Hola desde el principal");
    }
}
```

Derivamos nuestra clase de la clase Thread, que heredará de su padre sus métodos y atributos.

Sobreescribimos el método `run()` con lo que queramos que realice el hilo.

El hilo se arranca llamando al método `start()` de la clase (heredado del padre).

Programación de Servicios y Procesos

2.2.-Creación de Hilos

Ejemplo extends Thread 2

```
public class MiThread1 extends Thread{
    public MiThread1(){
        this.start();
    }
    public void run(){
        System.out.println("Soy el hilo. Voy a contar.");
        for (int x=1; x<=100; x++)
            System.out.println(x);
    }
}
public class Principal {
    public static void main(String[] args) {
        MiThread1 t1=new MiThread1();
        System.out.println("Hola desde el principal");
    }
}
```

La llamada al método start se puede realizar desde el main (ejemplo anterior) o desde el propio constructor de la clase (este ejemplo). En el segundo caso, el hilo se arrancará en cuanto hagamos el new de la clase.

2:-La interfaz Runnable

Forma general del cuerpo de una clase hilo mediante extensión de la clase Thread:

```
public class NombreClase implements runnable{  
    //propiedades, constructores y métodos de la clase  
    ....  
    public void run(){  
        // acciones que lleva a cabo el hilo  
        ....  
    }  
}
```

Dentro de la clase que usa a NombreClase, la siguiente es la forma del código para crear el hilo e iniciar su ejecución mediante el llamado al método start():

```
NombreClase unHilo = new NombreClase();  
.  
.  
unHilo.start();
```

Programación de Servicios y Procesos

2.2.-Creación de Hilos

Ejemplo Implements Runnable 1

```
public class MiThread2 implements Runnable {
    public void run() {
        System.out.println("Soy el hilo creado. Voy a contar.");
        for (int x=1; x<=10; x++)
            System.out.println(x + " ");
    }
}

public class Principal {
    public static void main(String[] args) {
        MiThread2 miT2 = new MiThread2();
        Thread t2 = new Thread(miT2);
        t2.start();
        System.out.println("Hola desde el principal");
    }
}
```

La instancia de la clase Thread la creamos en el main, pasándole como parámetro un objeto de nuestra clase. El hilo se arranca llamando al método start de la instancia de la clase Thread desde el main.

Programación de Servicios y Procesos

2.2.-Creación de Hilos

Ejemplo Implements Runnable 2

```
public class MiThread2 implements Runnable{
    public MiThread2(){
        Thread thread2=new Thread(this);
        thread2.start();
    }
    public void run(){
        System.out.println("Soy el hilo. Voy a contar.");
        for (int x=1; x<=10; x++)
            System.out.print(x+ " ");
    }
}

public class Principal {
    public static void main(String[] args) {
        MiThread2 mit2=new MiThread2();
        System.out.println("Hola desde el principal");
    }
}
```

Hacemos que nuestra clase implemente la interfaz Runnable. Implementamos el método run() con lo que queramos que realice el hilo. Se debe crear una instancia de la clase Thread dentro de nuestra clase, la cual representará el hilo a ejecutar. El hilo se arranca llamando al método start() de la instancia de la clase Thread desde el constructor de nuestra clase.

Es muy importante tener presente que para que los hilos sean ejecutados concurrentemente tenemos que invocar al método **start** y este método invocará al método **run**.

¿Qué sucederá si en vez de invocar al método start invocamos directamente al método run?

Que el programa será secuencial y los métodos run de cada hilo se ejecutarán secuencialmente en el orden que fueron invocados.

Join

Se emplea el método join de la clase Thread para que el main espere a que finalice el hilo para proseguir la ejecución.

```
public static void main()
{
    MiThread2 miT2=new MiThread2();
    miT2.start();
    miT2.join();
    System.out.println("Hola desde el principal");
}
```